

CodeColosseum

Software Design Specification

Version 1.0

Kerem Adalı 150123055

Buğra Kaya 150123045

Ali Talip Keleş 150122029

Prepared for

CSE3044 Software Engineering Term Project

Marmara University, Spring 2025–2026

April 2026

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Software Context	3
1.4	Major Constraints	3
1.5	Definitions, Acronyms, and Abbreviations	4
1.6	References	4
2	Design Considerations	4
2.1	Design Assumptions and Dependencies	4
2.2	General Constraints	5
2.3	System Environment	5
2.4	Development Methods	5
3	Architectural and Component-Level Design	6
3.1	System Structure	6
3.1.1	Architectural Style	6
3.1.2	Architecture Diagram	6
3.1.3	Data Flow Diagram	8
3.2	Component Descriptions	10
3.2.1	UserService	10
3.2.2	MatchmakingService	11
3.2.3	MatchService	12
3.2.4	JudgeService	14
3.2.5	RatingService	15
3.2.6	ProblemService	16
3.2.7	LeaderboardService	16
4	UML Diagrams	17
4.1	Class Diagram	17
4.2	Sequence Diagrams	18
4.2.1	User Registration and Login	18
4.2.2	Matchmaking Flow	20
4.2.3	Match Session and Code Submission	22
4.2.4	Problem Proposal Approval	23
4.2.5	Rating Update	25
5	Data Model	27
5.1	Entity Descriptions	27
5.2	ERD (Entity Relationship Diagram)	28
5.3	Entity-Relationship Overview	29
6	Restrictions, Limitations, and Constraints	29
7	Conclusion	30
A	Decision Log	30
A.1	Decision Log 1: Code Execution Strategy	30
A.2	Decision Log 2: Rating Algorithm	31
A.3	Decision Log 3: Match Winner Determination	31
A.4	Decision Log 4: Persistence Strategy	32

A.5	Decision Log 5: Real-time Communication Protocol	32
A.6	Decision Log 6: Authentication Mechanism	33
A.7	Decision Log 7: Matchmaking Queue Implementation	33
A.8	Decision Log 8: Matchmaking Proximity Threshold and Scan Interval . . .	34
A.9	Decision Log 9: Frontend Framework	34

1 Introduction

This section is reproduced and adapted from the CodeColosseum Software Requirements Specification (SRS) to provide the necessary background for the design decisions documented in the sections that follow.

1.1 Purpose

This Software Design Specification (SDS) defines the architectural and component-level design of **CodeColosseum**, a real-time 1v1 competitive programming platform. The document translates the functional and non-functional requirements stated in the SRS into a concrete technical blueprint that guides implementation. It covers the system architecture, major software components and their responsibilities, all inter-component interfaces, the data model, and the key design decisions taken during the design phase, together with the alternatives that were considered and rejected.

The intended audience is the development team, who use this document as the authoritative design reference; the course evaluators, who assess conformance to software engineering practice; and any future maintainer who needs to understand the structure of the system.

1.2 Scope

CodeColosseum is a web-based platform in which two registered users compete head-to-head in real time by solving the same algorithmic problem simultaneously. The system encompasses user authentication, Elo-based matchmaking, live match session management, sandboxed code evaluation via the Judge0 API, a community problem proposal workflow, and a public global leaderboard with private per-user match history.

The design described in this document covers the complete system as scoped in the SRS. Features explicitly out of scope — team contests, in-match chat, plagiarism detection, and spectator modes — are not addressed.

1.3 Software Context

CodeColosseum is an independent, self-contained web application with no dependencies on institutional infrastructure. It is inspired by Codeforces and LeetCode but is differentiated by its exclusive focus on synchronous 1v1 real-time competition. The system is scoped for an academic user base of university students and software developers.

1.4 Major Constraints

- **External execution dependency:** All code execution is delegated to the Judge0 REST API. The application server must never execute user-submitted code directly.
- **Technology stack:** Backend in Java 17 / Spring Boot 3.x; frontend in React 18; database in PostgreSQL 14+.
- **Team and timeline:** Three developers, one academic semester.
- **Browser support:** Current stable releases of Chrome, Firefox, and Edge; minimum viewport width of 1024 px.

1.5 Definitions, Acronyms, and Abbreviations

Term	Definition
SDS	Software Design Specification
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
Elo	Skill rating method; see FR-5 for the formula
K-factor	Constant controlling Elo rating volatility (fixed at 32)
Judge	Subsystem evaluating code submissions against test cases
Sandbox	Isolated execution environment preventing host access
AC	Accepted — all test cases pass
WA	Wrong Answer
TLE	Time Limit Exceeded
MLE	Memory Limit Exceeded
CE	Compilation Error
RE	Runtime Error
PSPEC	Processing Specification (narrative for a component)
DTO	Data Transfer Object
SPA	Single-Page Application
WS	WebSocket
DFD	Data Flow Diagram
FSM	Finite State Machine

1.6 References

1. IEEE Std 830-1984, *IEEE Guide to Software Requirements Specifications*.
2. IEEE Std 1016-2009, *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*.
3. Judge0 API Documentation. <https://judge0.com>
4. Spring Boot Reference Documentation. <https://spring.io/projects/spring-boot>
5. React Documentation. <https://react.dev>
6. PostgreSQL 14 Documentation. <https://www.postgresql.org/docs/>
7. E. W. Elo, *The Rating of Chessplayers, Past and Present*, Arco Publishing, 1978.
8. CodeColosseum Software Requirements Specification, Marmara University CSE3044, March 2026.

2 Design Considerations

2.1 Design Assumptions and Dependencies

The following assumptions were made during the design phase. If any assumption proves incorrect, the affected design decisions will need to be revisited.

1. The Judge0 v2 API will remain publicly accessible and free of charge for the submission volumes expected during the academic evaluation period.

2. All users access the platform via a JavaScript-enabled modern browser. No non-JavaScript or server-side rendering fallback is required.
3. A PostgreSQL 14+ server is co-located with or network-accessible from the application server throughout development and evaluation.
4. The standard Elo formula with $K=32$ is an adequate skill proxy for the expected volume of matches per user. Initial ratings start at 1000.
5. The Spring Boot embedded servlet container (Tomcat) is sufficient to serve both the REST API and WebSocket upgrade requests without a separate reverse proxy during evaluation.

2.2 General Constraints

Hardware / OS environment. The backend must run on any OS supporting Java 17. No specific hardware beyond standard server hardware is assumed.

Security. Passwords must be bcrypt-hashed (cost ≥ 10). All API endpoints performing state-changing operations must require a valid JWT. User-submitted code must never be executed outside the Judge0 sandbox.

Performance. 95% of submission verdicts must be returned within 10 seconds for up to 20 concurrent active matches. Leaderboard and dashboard pages must load within 2 seconds for up to 100 concurrent users.

Maintainability. Code execution logic must be hidden behind a `JudgeService` interface. Replacing Judge0 must require changes only to that one implementation class.

Portability. The frontend must function identically on Chrome, Firefox, and Edge without browser-specific CSS or JavaScript.

Data retention. Match, Submission, and RatingHistory records are retained indefinitely. Rejected problem proposals are retained for audit purposes.

2.3 System Environment

Component	Technology
Frontend	React 18, JavaScript (ES2022), browser WebSocket API
Backend	Java 17, Spring Boot 3.x, Spring Security
Database	PostgreSQL 14+
Auth tokens	JSON Web Tokens (JJWT library)
Real-time comms	STOMP over WebSocket (Spring WebSocket)
Code execution	Judge0 v2 REST API (external, HTTPS)
Build tools	Maven (backend), npm / Vite (frontend)
Version control	Git

2.4 Development Methods

The backend follows a strict **three-layer architecture** (NFR-12):

- **Controller layer** — handles HTTP/WebSocket requests, validates DTOs, and delegates to the service layer. Contains no business logic.
- **Service layer** — contains all business logic: matchmaking algorithms, Elo calculation, match resolution, and problem approval rules.

- **Repository layer** — wraps all database access. No SQL appears in service or controller classes.

Execution of user code is abstracted behind a **JudgeService** interface (NFR-11), allowing the Judge0 implementation to be swapped without modifying any other class.

The frontend is a single-page application. It communicates with the backend exclusively via a versioned REST API for standard interactions and a STOMP-over-WebSocket connection for real-time events (timer synchronisation, verdict delivery, match outcome).

3 Architectural and Component-Level Design

3.1 System Structure

CodeColosseum adopts a **client-server architecture** with a clear separation between the browser-side SPA and the Spring Boot application server. The server, in turn, communicates with two external dependencies: the PostgreSQL database and the Judge0 API. No component communicates with another except through the backend; in particular, the frontend never contacts Judge0 directly.

3.1.1 Architectural Style

The backend implements the **layered architecture** pattern with three tiers (Controller → Service → Repository) complemented by an **interface-driven design** for the execution subsystem. The overall deployment topology is a **two-tier client-server** model: one server process handles all backend concerns, and one browser-side SPA handles all presentation concerns.

Within the server, the matchmaking subsystem uses an **in-memory queue** with a periodic scan strategy rather than a full event-driven broker, which is appropriate for the expected concurrency levels (tens of simultaneous users).

3.1.2 Architecture Diagram

Figure 1 presents the UML component diagram for CodeColosseum. It shows the four principal components of the system — the React Frontend, the Spring Boot Backend, the PostgreSQL database, and the Judge0 external API — together with the interfaces that connect them.

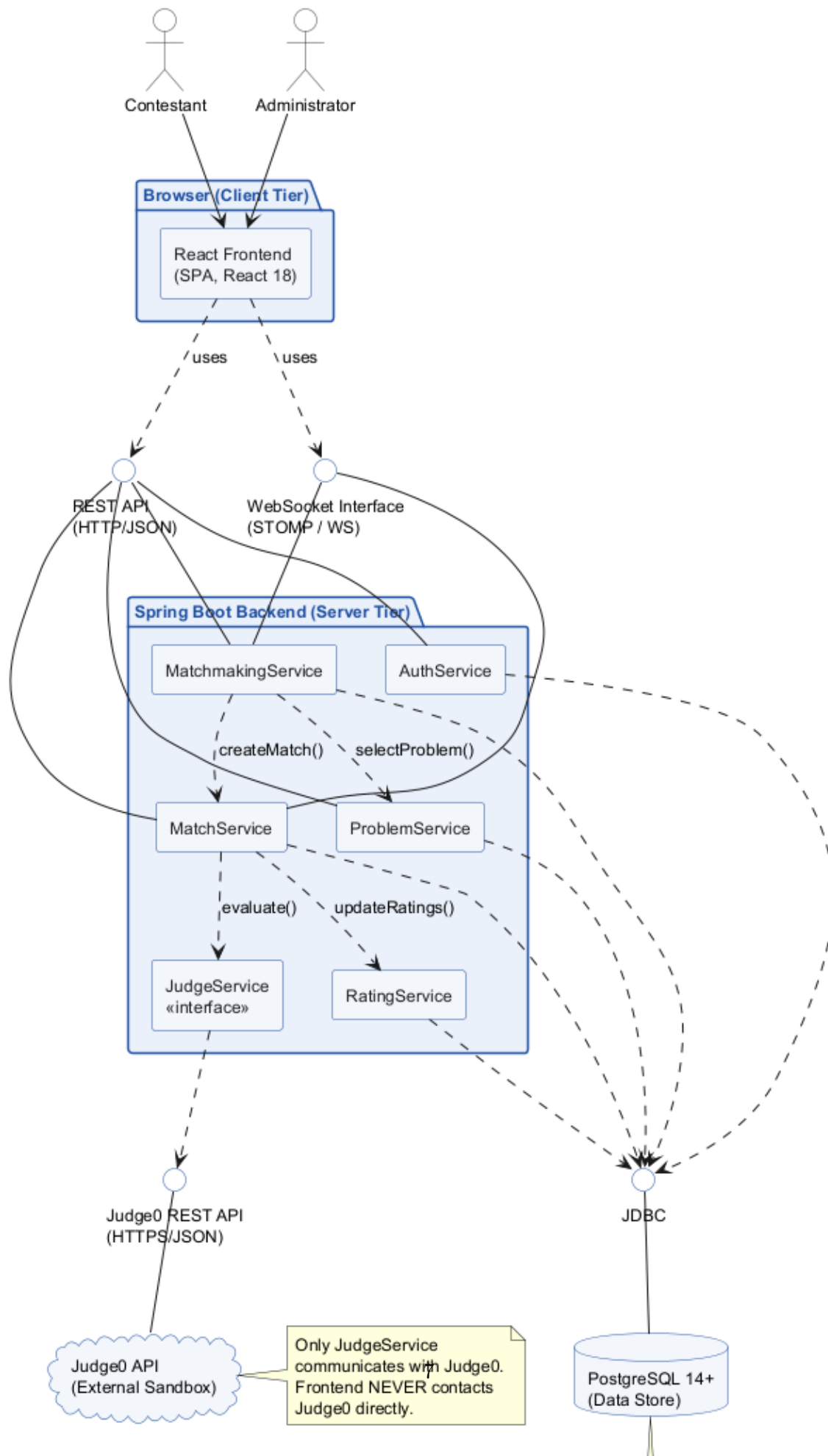
CodeColosseum – System Architecture (Component Diagram)

Diagram explanation. The **React Frontend** is a browser-resident SPA that exposes no interfaces to external actors; it is a pure consumer. It connects to the backend through two provided interfaces: the *REST API* (HTTP/HTTPS, JSON payloads) for all standard request-response interactions such as login, registration, problem proposal submission, and leaderboard queries; and the *WebSocket Interface* (STOMP over WS/WSS) for all real-time push events such as match-found notifications, problem delivery, verdict updates, and match outcome announcements.

The **Spring Boot Backend** is the architectural hub. It provides the REST API and the WebSocket interface consumed by the frontend, and it in turn consumes two further interfaces: the *JDBC interface* to PostgreSQL for all persistent storage, and the *Judge0 REST API* (HTTPS, JSON) for sandboxed code execution. The backend is internally decomposed into six named sub-components — *AuthService*, *MatchmakingService*, *MatchService*, *JudgeService*, *RatingService*, and *ProblemService* — each of which is described in detail in the sections that follow.

The **PostgreSQL Database** is a passive external datastore. It is accessed exclusively by the backend through the JDBC interface and is never contacted directly by the frontend or by Judge0.

The **Judge0 API** is an external third-party service. The backend's *JudgeService* sends HTTP POST requests containing source code, language identifiers, and resource limits; Judge0 returns per-test-case verdicts in JSON. The frontend is architecturally prohibited from reaching Judge0.

3.1.3 Data Flow Diagram

While the component diagram shows *which* elements of the system are connected, it does not describe *what* information travels between them or *how* that information is transformed along the way. Figure 2 provides this complementary view in the form of a Level 1 Data Flow Diagram (DFD). It identifies the external entities that produce and consume data (the Contestant, the Administrator, and Judge0), the six principal processes implemented by the backend, and the six persistent data stores (one per primary database table).

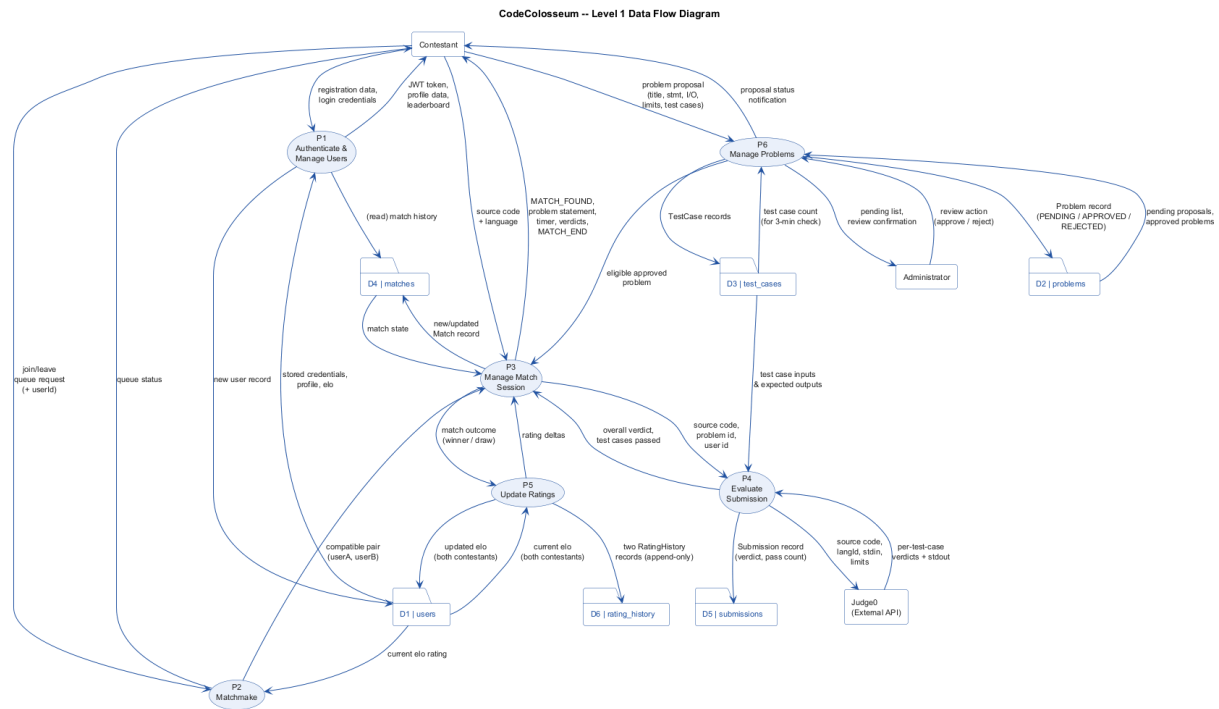


Figure 2: Level 1 Data Flow Diagram — Principal System Processes and Data Stores

Diagram explanation. The diagram uses standard Gane-Sarson notation: rectangles denote external entities, rounded rectangles denote processes, and open-ended bars denote persistent data stores. Each labelled arrow represents a data flow whose label identifies the content being transferred (not the mechanism, which is covered in the component diagram).

Five processes read from or write to the *users* data store. The *Authenticate & Manage Users* process (P1) receives registration data and login credentials from the Contestant, writes new **User** records, reads stored password hashes during login, and returns a JWT on successful authentication. The same process services public profile and leaderboard reads.

The *Matchmake* process (P2) receives “join queue” requests (carrying the Contestant’s user ID), reads the Contestant’s current Elo rating from the *users* store to perform proximity comparison, and — when a compatible pair is identified — delegates to the *Manage Match Session* process (P3). P3 writes a new **Match** record to the *matches* store, reads an eligible approved problem from the *problems* store via the *Manage Problems* process (P6), and emits a “match found” notification back to both Contestants.

During an active match, P3 receives code submissions from the Contestant and forwards them to the *Evaluate Submission* process (P4), which owns all interaction with the external Judge0 entity. P4 reads the problem’s test cases from the *test_cases* store (these are never exposed externally), transmits source code and stdin to Judge0, receives per-test-case verdicts back, aggregates them into a single overall verdict, writes a **Submission** record to the *submissions* store, and returns the verdict to P3. P3 then applies its win/loss/continue logic described in section 3.2.3.

On match completion, P3 triggers the *Update Ratings* process (P5), which reads both contestants’ current Elo ratings from *users*, applies the Elo formula, writes updated ratings

back to *users*, and appends two audit records to the *rating_history* store. The rating deltas are returned via P3 as part of the final “match end” payload delivered to both Contestants.

The *Manage Problems* process (P6) handles the parallel proposal workflow: Contestants submit new problems (written as **PENDING** records into *problems* and *test_cases*), and the Administrator reads pending proposals and writes approve/reject decisions (updating the **status** column in the *problems* store). Only approved problems are visible to P3 for matchmaking use.

Two invariants are visible in the diagram and are preserved throughout the design: first, all flows to and from Judge0 pass through P4, never through any other process or directly from the Contestant; second, flows from the *test_cases* store terminate at P4 and never exit the system boundary, protecting the privacy of expected outputs.

3.2 Component Descriptions

The six service components of the backend are described individually below. Each subsection follows the IEEE 1016 component template: processing narrative (PSPEC), interface description, and processing detail including class hierarchy, restrictions, performance considerations, and design constraints.

3.2.1 UserService

Processing Narrative (PSPEC). **UserService** is responsible for all user account lifecycle operations: registration of new accounts, credential validation on login, and JWT issuance. It enforces uniqueness constraints on both username and email address, delegates password hashing to the BCrypt encoder, and initialises the default Elo rating (1000) and role (CONTESTANT) for every new account. It also exposes read-only profile and leaderboard endpoints for authenticated and public access.

Interface Description.

Endpoint	Method	Auth	Description
/api/user/register	POST	None	Accepts username, email, password; creates account; returns 201
/api/user/login	POST	None	Validates credentials; returns JWT token
/api/user/me	GET	JWT	Returns the authenticated user’s own profile
/api/user/{username}	GET	None	Returns the public profile of the specified user
/api/user/leaderboard	GET	None	Returns all users ranked by Elo rating in descending order

Processing Detail. Class hierarchy:

- **AuthController** (Controller layer) — validates incoming DTOs, delegates to **AuthService**, returns HTTP responses.
- **AuthService** (Service layer) — orchestrates registration and login logic.
- **UserRepository** (Repository layer) — Repository for the **User** entity; provides **findByEmail** and **existsByUsername**.
- **JwtUtil** (Utility) — generates and validates JWT tokens using the JJWT library.
- **User** (Entity) — maps to the **users** table.

Restrictions: Usernames must be 3–20 alphanumeric characters or underscores, unique system-wide. Passwords must be at least 8 characters and must never be stored in plain-text. Email addresses must conform to RFC 5322 and be unique system-wide.

Performance: BCrypt hashing at cost factor 10 is intentionally slow (approx. 100 ms per hash on commodity hardware) to resist brute-force attacks. This is acceptable because authentication is not a high-frequency operation.

Design constraints: Spring Security is used for password encoding and authentication filter configuration. The JWT secret is externalised to application configuration and must not be hard-coded.

3.2.2 MatchmakingService

Processing Narrative (PSPEC). **MatchmakingService** maintains an in-memory queue of contestants who are waiting for an opponent. Every 1000 ms a scheduled task scans the queue for pairs of users whose Elo ratings differ by no more than 200 points (configurable). When a compatible pair is found, the service atomically removes both from the queue, selects a suitable problem (excluding problems either contestant has encountered before), persists a new **Match** record with status **ACTIVE**, and notifies both users via WebSocket.

Interface Description.

Endpoint	Method	Auth	Description
/api/queue/join	POST	JWT	Adds authenticated user to the matchmaking queue
/api/queue/leave	POST	JWT	Removes authenticated user from the queue
WS topic: — /topic/match.found.{userId}	—	WS session	Push notification carrying matchId when a match is created

Processing Detail. Class hierarchy:

- **QueueController** — exposes the REST join/leave endpoints.
- **MatchmakingService** — owns the **ConcurrentLinkedQueue** of **QueueEntry** objects; runs the scheduled scan.
- **QueueEntry** (value object) — holds **userId**, **eloRating**, and **joinedAt** timestamp.
- **MatchService** — called by **MatchmakingService** to persist the new match and select the problem.

- **WebSocketNotificationService** — sends STOMP messages to individual user topics.

Restrictions: A user already in the queue or in an active match must not be permitted to join again (HTTP 409). A user must not be matched against themselves. If no eligible problem exists for a given pair, the match is cancelled with no Elo change.

Performance: The queue scan runs every 1000 ms on a Spring `@Scheduled` thread. At expected concurrency levels (tens of users), the scan completes in under 1 ms per cycle.

Design constraints: The queue is in-memory and not replicated. This is acceptable for the single-node academic deployment. A future distributed deployment would require a Redis-backed queue.

3.2.3 MatchService

Processing Narrative (PSPEC). **MatchService** manages the full lifecycle of a match session from creation to resolution. On creation it records both contestants and the selected problem. During the session it receives submission results from **JudgeService** and determines whether the match should conclude immediately (first AC verdict) or whether it should wait for the timer to expire. On conclusion it updates the match record to **COMPLETED**, applies the tie-breaking rule if necessary (most test cases passed across all submissions; draw on equality), and triggers **RatingService**.

Interface Description.

Endpoint	Method	Auth	Description
/api/match/{id}	GET	JWT	Returns match state (problem, timer, status)
/api/match/{id}/submit	POST	JWT	Accepts a code submission for evaluation
WS topic: — /topic/match.{matchId}		WS session	Push events: match-start, verdict, match-end

Processing Detail. Class hierarchy:

- **MatchController** — REST and WebSocket controller for match interactions.
- **MatchService** — business logic: session lifecycle, timer management, win condition evaluation.
- **Submission (Entity)** — maps to the `submissions` table.
- **MatchRepository, SubmissionRepository** (Repositories).

Restrictions: A contestant may not view their opponent's code or verdict during an active match. Match duration is fixed at 30 minutes. There is no submission count limit. A contestant who disconnects for more than 5 minutes forfeits.

Performance: Verdict delivery must meet the 10-second SLA (NFR-1). The timer is maintained server-side and pushed to clients at regular intervals via WebSocket to prevent client-side manipulation.

Match Session State Machine. Because the behaviour of `MatchService` is driven by a non-trivial set of discrete events (submission verdicts, timer expiry, disconnections) and must honour strict transition rules — for example, once a match has been resolved no further submission may alter its outcome — the component’s lifecycle is formalised as a finite-state machine in Figure 3.

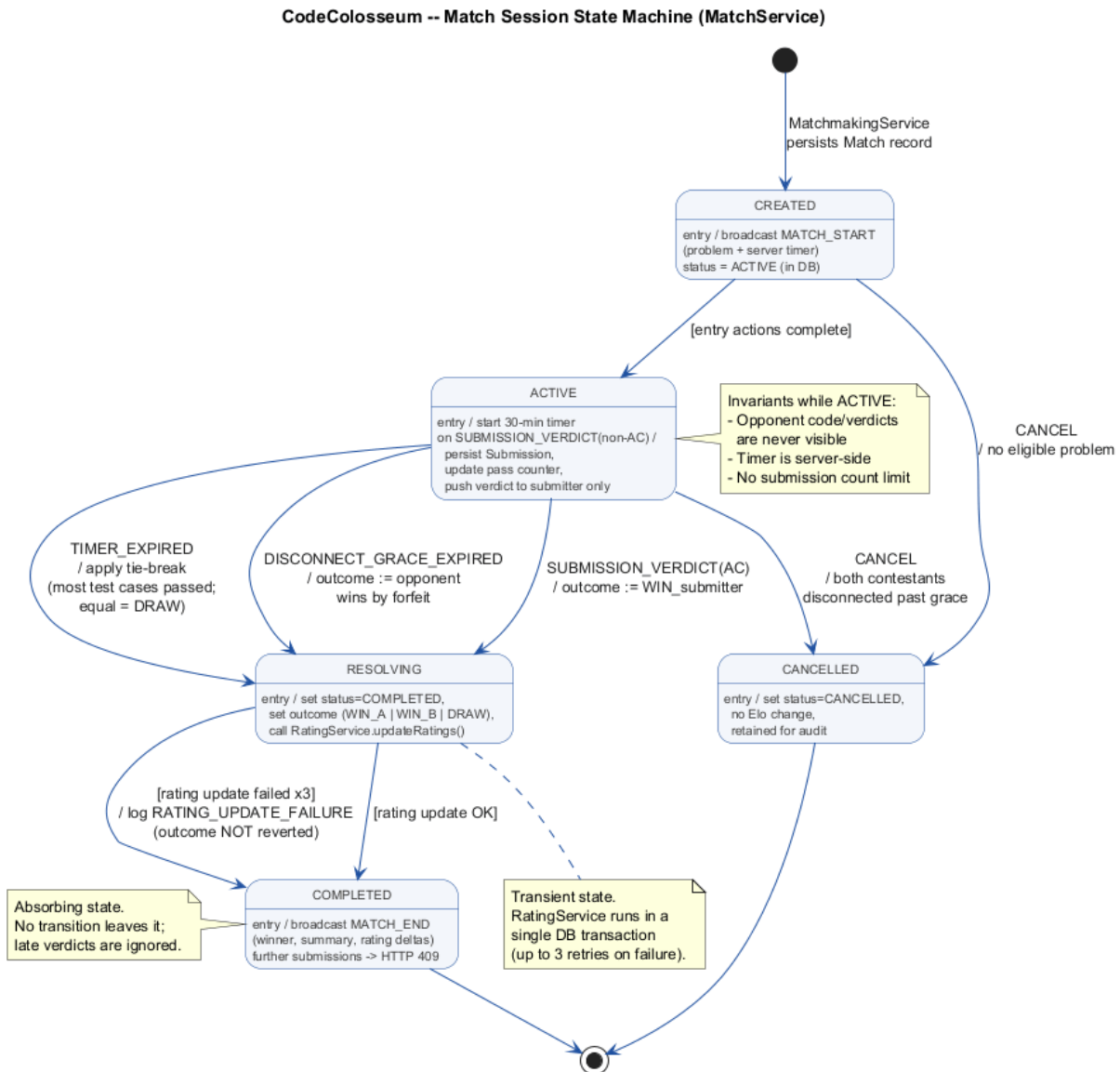


Figure 3: UML State Machine Diagram — Match Session Lifecycle

Diagram explanation. The diagram enumerates five states reachable by any `Match` instance. Every match begins in the **CREATED** pseudo-state the moment `MatchmakingService` persists the record; this state is transient and exists only to bracket the *entry* action (broadcast of the `MATCH_START` WebSocket event containing the problem statement and the synchronised server timer). On completion of that entry action the state machine transitions unconditionally to **ACTIVE**.

The **ACTIVE** state is the steady state of a live match. While in this state, the machine responds to four distinct events. (i) A `SUBMISSION_VERDICT(AC)` event — triggered

when `JudgeService` returns a verdict of `Accepted` for either contestant — causes a transition to **RESOLVING** with the outcome set to `WIN_A` or `WIN_B` according to which contestant submitted. (ii) A `SUBMISSION_VERDICT(non-AC)` event is handled by an internal self-transition: the submission is persisted, the test-case pass count is updated on the per-contestant accumulator, the verdict is delivered to the submitting contestant only, and the match remains **ACTIVE**. (iii) A `TIMER_EXPIRED` event causes a transition to **RESOLVING**; during this transition the tie-breaking rule is evaluated (contestant with more test cases passed wins; equal counts produce a `DRAW`). (iv) A `DISCONNECT_GRACE_EXPIRED` event — fired after a contestant has been disconnected for more than five minutes without reconnecting — transitions to **RESOLVING** with the outcome set to the opponent's win by forfeit.

The **RESOLVING** state is a brief transient state whose entry action updates the match record to **COMPLETED**, sets the final `outcome` field, and invokes `RatingService.updateRatings()` synchronously within the same transaction. On successful completion of these actions the machine transitions unconditionally to **COMPLETED**. If rating update fails after three retries, the machine still transitions to **COMPLETED** but logs a `RATING_UPDATE_FAILURE` event for manual reconciliation; the match outcome is not reverted.

The **COMPLETED** state is terminal for the normal happy path. On entry, the state machine broadcasts the `MATCH_END` event to both contestants carrying the winner, final verdict summary, and rating deltas. Any further submission attempt for a match in this state is rejected by `MatchController` with HTTP 409.

The **CANCELLED** state is the alternative terminal state, reachable from **CREATED** or **ACTIVE** via a `CANCEL` event. Cancellation occurs only under two well-defined conditions: no eligible problem could be selected for the pair (cancellation from **CREATED** with no Elo change), or both contestants disconnect beyond the grace period (cancellation from **ACTIVE** with no Elo change). Cancelled matches are retained for audit but do not appear in contestant match history pages.

Three invariants hold across the entire machine. First, the **COMPLETED** and **CANCELLED** states are absorbing: no transition leaves them. Second, rating updates occur exactly once per match, on the transition into **COMPLETED**. Third, all transitions out of **ACTIVE** terminate either in **RESOLVING** (from which **COMPLETED** is unconditional) or in **CANCELLED**; no state re-entry to **ACTIVE** is permitted, guaranteeing that a resolved match cannot be inadvertently reopened by a late-arriving verdict.

3.2.4 JudgeService

Processing Narrative (PSPEC). `JudgeService` is the abstraction layer between the rest of the backend and the Judge0 API. For each submission, it iterates over the problem's test cases, posting each one as a separate execution request to Judge0 with the source code, language identifier, and problem-defined resource limits. It collects per-test-case verdicts, aggregates them into an overall verdict (`AC` if all pass; first non-`AC` otherwise), and records the test case pass count for tie-breaking. On Judge0 error it marks the submission as `JUDGE_ERROR` without voiding the match.

Interface Description. `JudgeService` is an internal interface with one concrete implementation (`Judge0ServiceImpl`). It is not directly exposed via REST; it is called

exclusively by `SubmissionService`.

Method signature	Description
<code>evaluate(code, langId, testCases, limits)</code>	Runs code against all test cases; returns <code>JudgeResult</code>

Processing Detail. Class hierarchy:

- `JudgeService` (interface) — defines the `evaluate` contract.
- `Judge0ServiceImpl` (implementation) — HTTP client logic for Judge0.
- `JudgeResult` (value object) — carries verdict, test cases passed, error message.
- `TestCase` (Entity) — problem-bound input/expected-output pair; never exposed to clients.

Restrictions: No user-submitted code may execute on the application server. Test case expected outputs are server-side only and must never be returned in any API response.

Performance: Each test case is a separate HTTP round-trip to Judge0. For problems with many test cases this may approach the 10-second SLA. The number of test cases per problem should be kept reasonable (recommended: 5–15) during the evaluation period.

Design constraints: The `Judge0ServiceImpl` is the only class that knows the Judge0 endpoint URL and API key. Replacing Judge0 requires only writing a new `JudgeService` implementation (NFR-11).

3.2.5 RatingService

Processing Narrative (PSPEC). `RatingService` computes and persists new Elo ratings for both contestants after every completed match. It is triggered automatically by `MatchService` when the match record is finalised. The update is executed within a single atomic database transaction covering both players' `User` rows and two new `RatingHistory` rows. If the transaction fails, it is retried up to three times; persistent failure is logged for manual review but does not revert the match outcome.

Processing Detail. The Elo formula applied is:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}, \quad R'_A = R_A + 32 \cdot (S_A - E_A)$$

where $S_A \in \{1, 0.5, 0\}$ for win, draw, and loss respectively. The formula is applied symmetrically for contestant B. The resulting rating is clamped to 0 from below (FR-5.5).

Class hierarchy:

- `RatingService` — implements the Elo calculation and DB transaction.
- `RatingHistory` (Entity) — records per-match rating changes.
- `UserRepository` — used to read and update both contestants' `eloRating` fields.

3.2.6 ProblemService

Processing Narrative (PSPEC). `ProblemService` handles the full problem lifecycle: proposal submission by contestants, review and approval or rejection by administrators, and retrieval of approved problems for use in matchmaking. It enforces the three-test-case minimum before an administrator is permitted to approve a proposal, and it notifies the proposing contestant of the decision via an in-system notification record.

Interface Description.

Endpoint	Method	Auth	Description
<code>/api/problems/propose</code>	POST	JWT (any)	Submit a new problem proposal
<code>/api/problems/pending</code>	GET	JWT (ADMIN)	List all PENDING proposals
<code>/api/problems/{id}/approve</code>	PATCH	JWT (ADMIN)	Approve a proposal
<code>/api/problems/{id}/reject</code>	PATCH	JWT (ADMIN)	Reject a proposal
<code>/api/problems/{id}</code>	GET	JWT (any)	Get a specific approved problem

Processing Detail. Class hierarchy:

- `ProblemController` — REST endpoints for proposal and review.
- `ProblemService` — business logic: validation, status transitions, notification dispatch.
- `Problem` (Entity) — maps to the `problems` table.
- `TestCase` (Entity) — child of `Problem`.
- `ProblemRepository`, `TestCaseRepository` (Repositories).

Restrictions: Only ADMIN role may approve or reject. A problem with fewer than three test cases cannot be approved (the endpoint returns HTTP 422). Test case inputs and expected outputs are never returned in any endpoint response visible to contestants.

3.2.7 LeaderboardService

Processing Narrative (PSPEC). `LeaderboardService` serves the global leaderboard and per-user match history. Leaderboard results are cached in-memory for 60 seconds to reduce database read load (FR-7.3). Match history is private; the service verifies that the requesting user's JWT subject matches the requested user ID before returning records.

Interface Description.

Endpoint	Method	Auth	Description
<code>/api/leaderboard</code>	GET	None (public)	Paginated global rankings

Endpoint	Method	Auth	Description
/api/users/{id}/history	GET	JWT (owner)	Paginated personal match history

4 UML Diagrams

4.1 Class Diagram

Figure 4 presents the design class diagram for CodeColosseum. It covers all entities and the principal service classes, together with their attributes, operations, and associations.

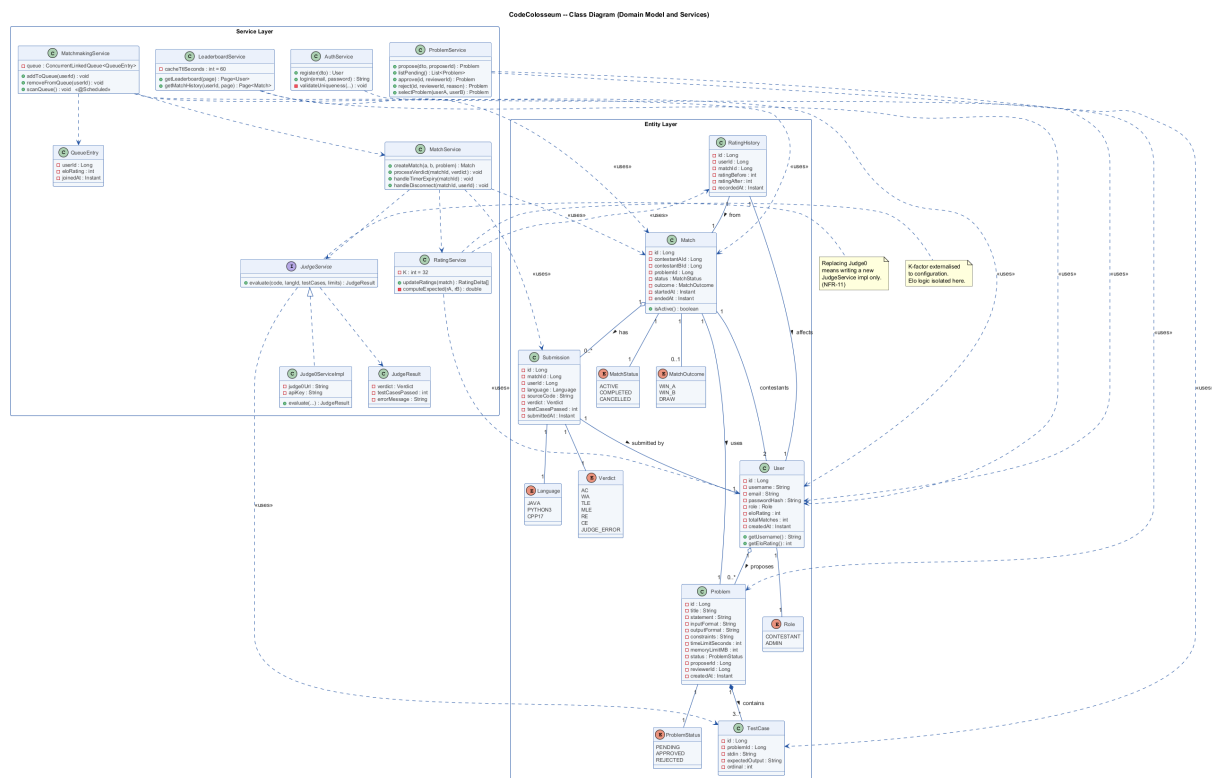


Figure 4: UML Class Diagram — Domain Model and Services

Diagram explanation. The **entity layer** forms the persistent core of the model. **User** is the central entity; it holds authentication credentials, the current Elo rating, and the assigned role. A **User** can propose zero or more **Problems** and participate in zero or more **Matches** (in either the role of contestant A or contestant B). Each **Match** is associated with exactly one **Problem** and generates zero or more **Submissions**, each linked to both the submitting **User** and the **Match**. A **Problem** owns one or more **TestCases**; test cases are never exposed to contestants. Every match conclusion produces exactly two **RatingHistory** records, one per contestant, forming an append-only audit trail of Elo changes.

The **service layer** classes are shown with dashed dependency arrows to the entities they govern. `AuthService` depends on `User`. `MatchmakingService` depends on `User` and `Match` and collaborates with `MatchService`. `JudgeService` (shown as an interface with

`Judge0ServiceImpl`) depends on `TestCase` and `Submission`. `RatingService` depends on `User` and `RatingHistory`. `ProblemService` depends on `Problem` and `TestCase`.

The multiplicity annotations are: a `User` may propose many `Problems` (0..*); a `Problem` has at least three `TestCases` (3..*); a `Match` involves exactly two `Users` (2) and exactly one `Problem` (1); a `Match` holds zero or more `Submissions` (0..*); each match end creates exactly two `RatingHistory` records.

4.2 Sequence Diagrams

The following sequence diagrams illustrate the dynamic behaviour of the system for the most architecturally significant interactions.

4.2.1 User Registration and Login

Figure 5 shows the message flow for account registration and subsequent login.

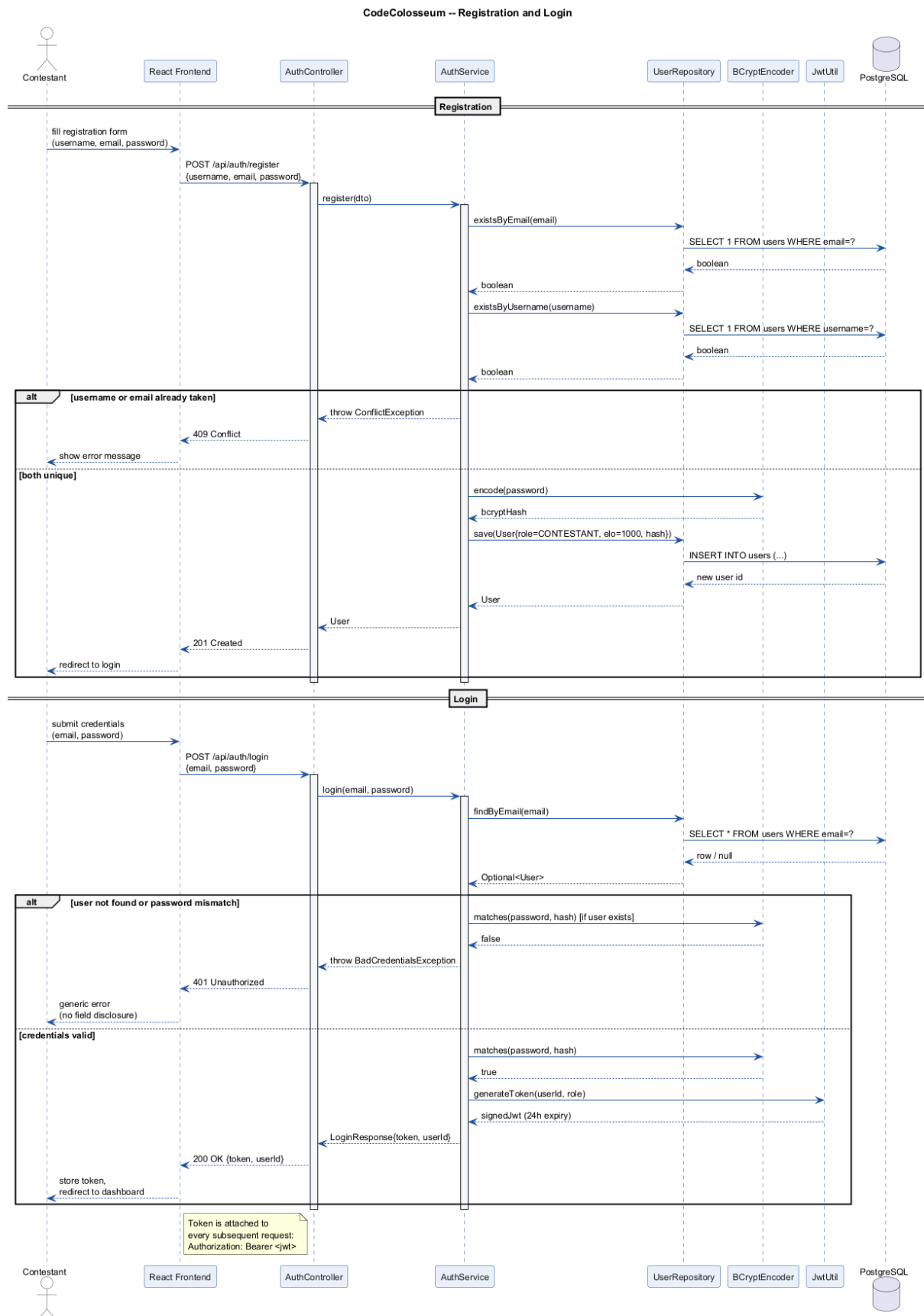


Figure 5: Sequence Diagram — User Registration and Login

Diagram explanation. The diagram is divided into two interaction frames. In the *Registration* frame, the contestant fills in the registration form and the browser sends a `POST /api/auth/register` request to the Spring Boot backend. `AuthController` forwards the request to `AuthService`, which first queries `UserRepository` to check that neither the email nor the username is already taken. If either is a duplicate, a 409 Conflict is returned immediately. If both are unique, `AuthService` encodes the password with BCrypt (cost 10) and persists the new `User` entity with role `CONTESTANT` and rating 1000. A 201 Created response is returned and the frontend redirects the user to the login screen.

In the *Login* frame, the contestant submits credentials via `POST /api/auth/login`. `AuthService` retrieves the `User` record by email, compares the submitted password against the stored BCrypt hash, and — on success — calls `JwtUtil` to generate a signed JWT with a 24-hour expiry. The token is returned to the frontend, stored client-side (e.g. in memory or `localStorage`), and attached to every subsequent request in the `Authorization: Bearer` header. Invalid credentials produce a generic 401 Unauthorized with no disclosure of which field was wrong.

4.2.2 Matchmaking Flow

Figure 6 shows the sequence from a contestant clicking “Find Match” to both players being redirected to the match screen.

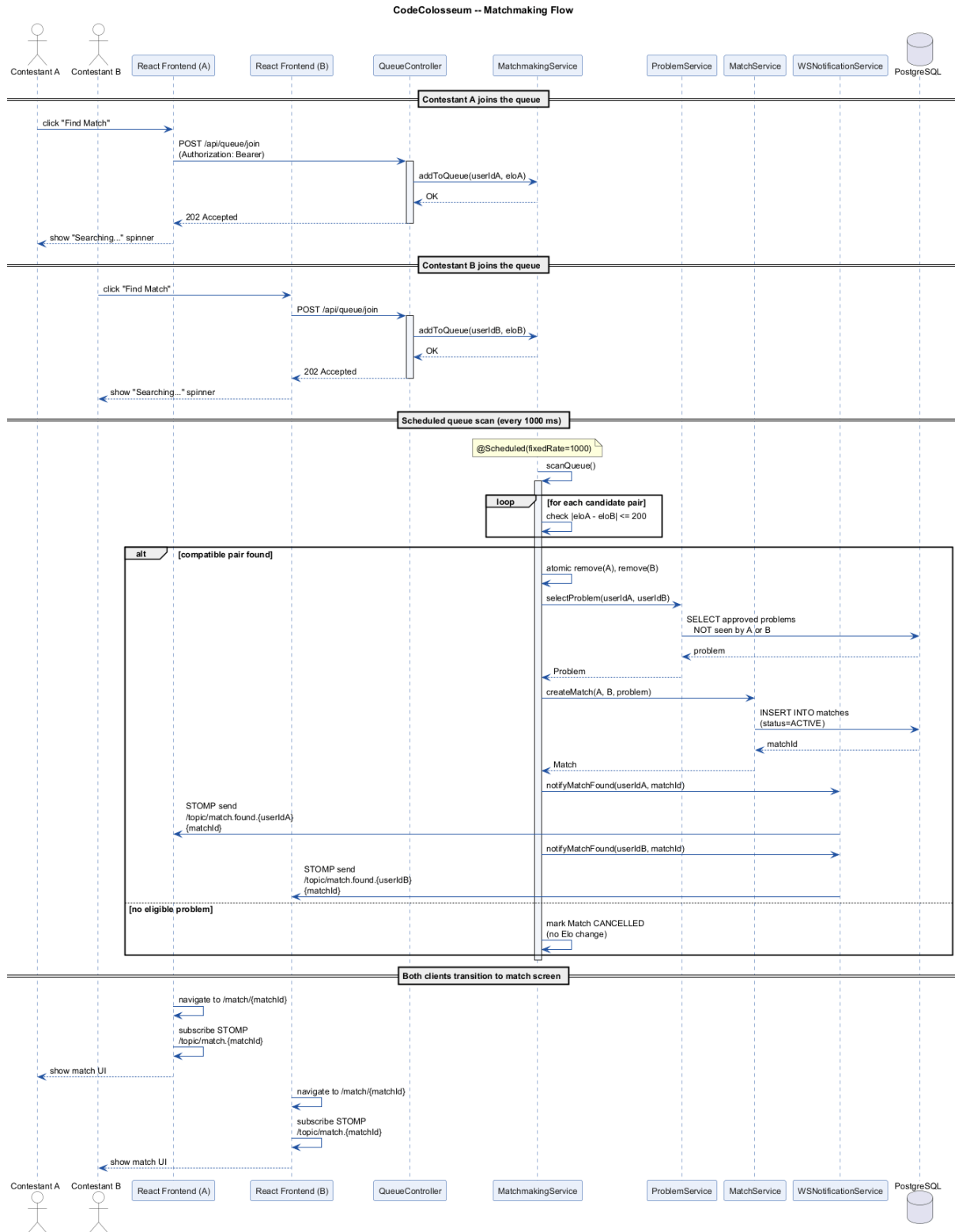


Figure 6: Sequence Diagram — Matchmaking

Diagram explanation. The contestant clicks the “Find Match” button, which triggers a `POST /api/queue/join` carrying the JWT. `QueueController` extracts the user ID from the token and calls `MatchmakingService.addToQueue()`, which adds a `QueueEntry` (`userId`, `eloRating`, `joinedAt`) to the `ConcurrentLinkedQueue`. The call returns immedi-

ately; the contestant's browser remains on the dashboard with a waiting indicator.

Concurrently, a Spring `@Scheduled` task fires every 1000 ms. It iterates the queue and checks all pairs for Elo proximity ($|\Delta\text{Elo}| \leq 200$). When a compatible pair is found, `MatchmakingService` atomically removes both entries, calls `ProblemService.selectProblem(userA, userB)` to pick an approved problem neither contestant has seen, and calls `MatchService.createMatch()` which persists a `Match` record with status `ACTIVE`. The service then invokes `WebSocketNotificationService` to push a `MATCH_FOUND` event carrying the `matchId` to each contestant's personal STOMP topic (`/topic/match.found.{userId}`). Upon receiving this event, both browsers navigate to the match screen and upgrade their WebSocket connection to subscribe to `/topic/match.{matchId}` for real-time match events.

4.2.3 Match Session and Code Submission

Figure 7 illustrates the full submission pipeline during a live match.

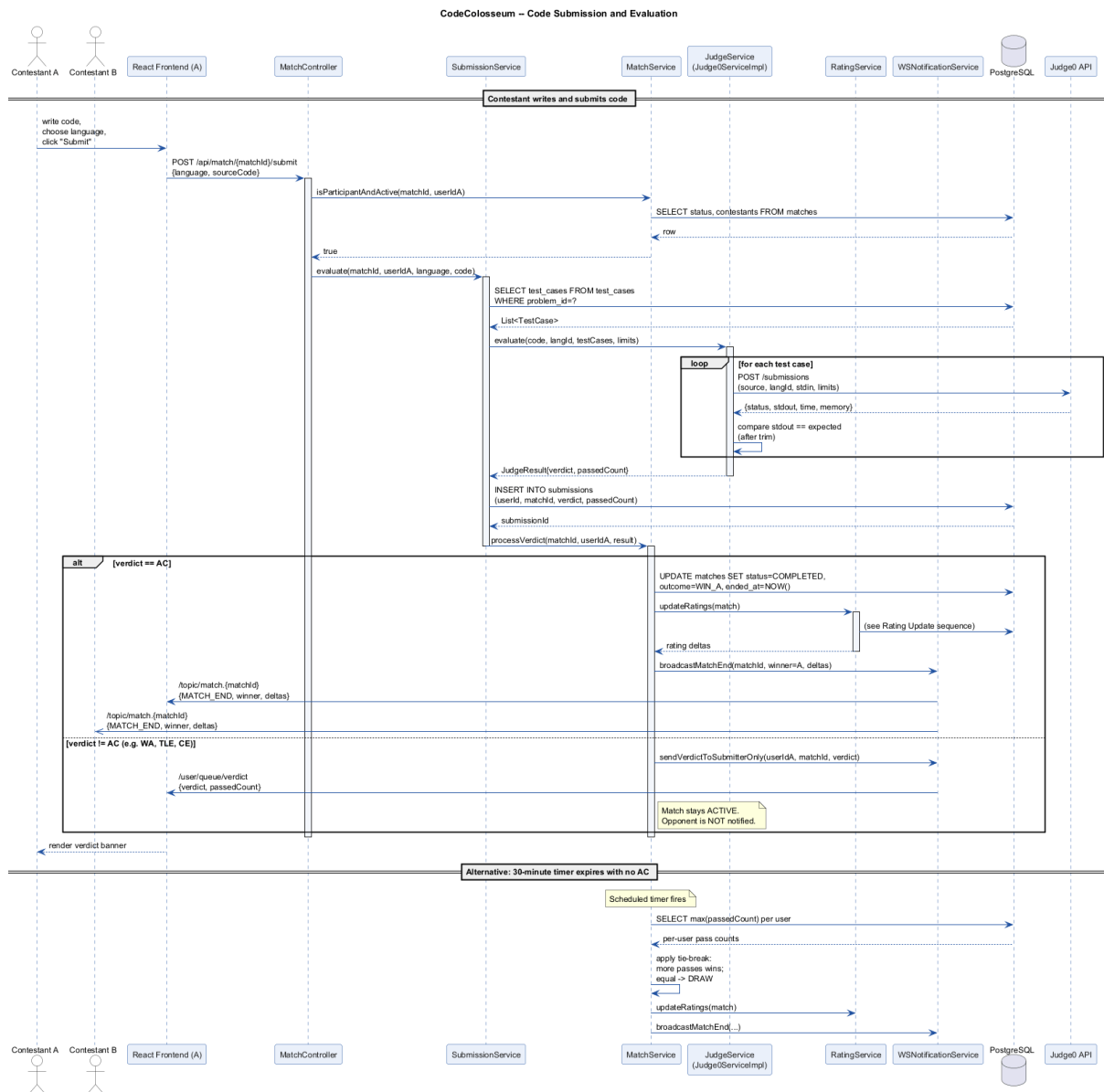


Figure 7: Sequence Diagram — Code Submission and Evaluation

Diagram explanation. Once on the match screen, the backend delivers the problem statement and starts the 30-minute countdown timer via WebSocket. The contestant writes code, selects a language, and clicks “Submit”. The browser sends a `POST /api/match/{matchId}/submit` with the source code and language identifier.

`MatchController` validates that the match is still `ACTIVE` and that the submitting user is a participant, then delegates to `SubmissionService.evaluate()`. This service retrieves the problem’s `TestCase` list from the database (inputs and expected outputs are never sent to the client) and calls `JudgeService.evaluate()`.

`Judge0ServiceImpl` iterates the test cases: for each one it posts a JSON execution request to the Judge0 REST API (source code, language ID, stdin, time limit, memory limit) and awaits the response. Each response carries a status token (accepted, wrong answer, TLE, etc.) and the program’s stdout. `Judge0ServiceImpl` compares stdout (trimmed) with the expected output and counts passing cases.

Once all test cases are evaluated, the aggregated `JudgeResult` is returned to `SubmissionService`, which persists a `Submission` record and calls `MatchService.processVerdict()`. If the verdict is `AC`, `MatchService` marks the `Match` as `COMPLETED`, records the winner, and calls `RatingService.updateRatings()`. It then broadcasts a `MATCH_END` WebSocket event to both contestants. If the verdict is not `AC`, only the submitting contestant receives the verdict update; the match continues. If the 30-minute timer expires before any `AC`, the tie-breaking rule (most test cases passed) is applied by `MatchService` and the match is likewise concluded.

4.2.4 Problem Proposal Approval

Figure 8 shows the sequence for a contestant submitting a problem proposal and an administrator approving it.

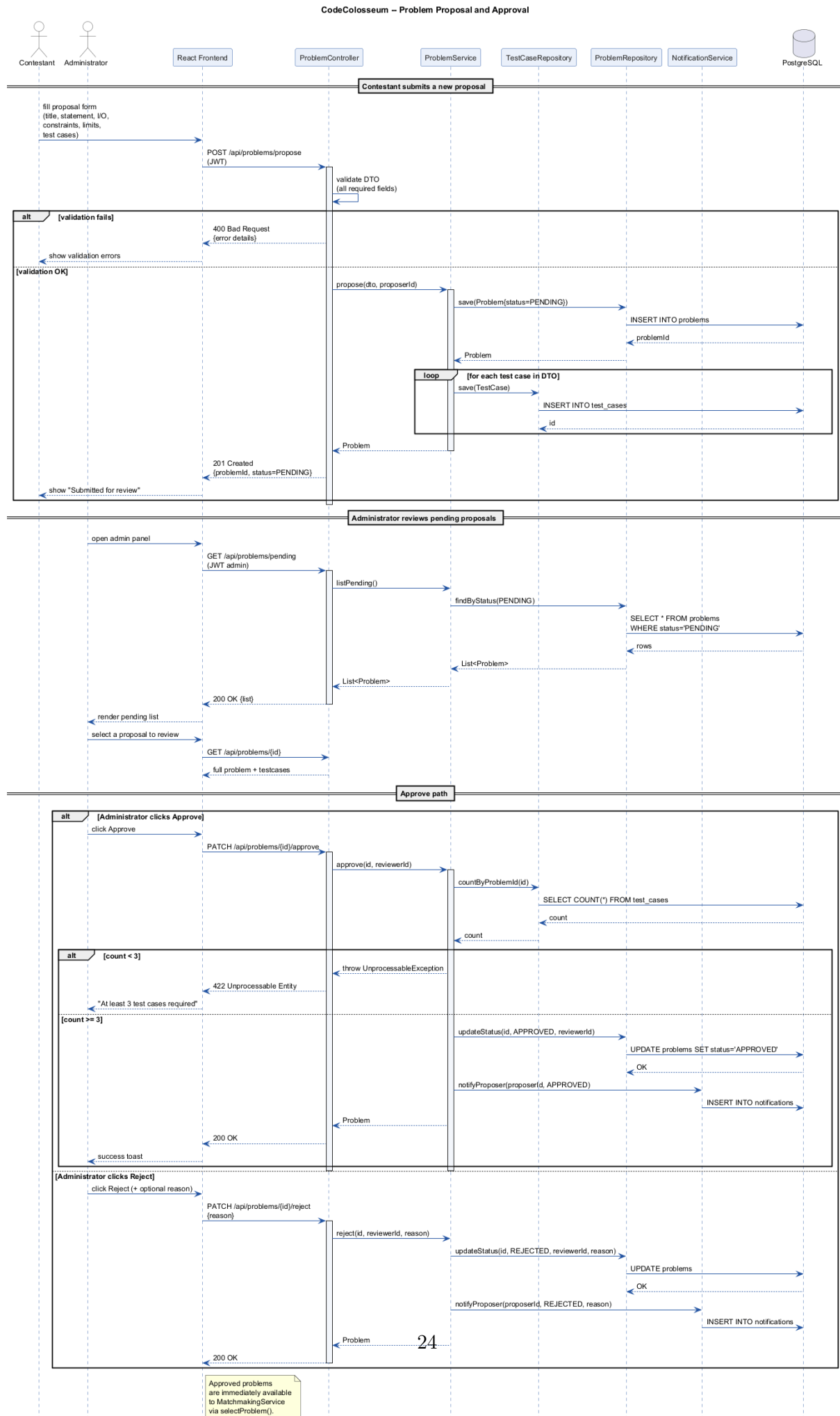


Diagram explanation. The contestant fills out the problem proposal form and submits it via `POST /api/problems/propose`. `ProblemController` validates all required fields (title, statement, I/O formats, constraints, time limit, memory limit, at least one test case pair). On success, `ProblemService` persists a `Problem` entity with status `PENDING` and associates the proposer’s user ID.

At a later point the administrator logs in, navigates to the admin panel, and calls `GET /api/problems/pending`, receiving a paginated list of pending proposals. The administrator selects one, optionally edits fields, and verifies that at least three test cases are present (the frontend disables the approve button otherwise). When the administrator clicks “Approve”, a `PATCH /api/problems/{id}/approve` is sent. `ProblemService` sets the status to `APPROVED`, making the problem immediately available in the active pool for matchmaking. An in-system notification record is created for the proposing contestant. If the administrator clicks “Reject” instead, the status is set to `REJECTED` and an optional rejection reason is stored and communicated to the proposer.

4.2.5 Rating Update

Figure 9 illustrates the automated rating update that follows every match conclusion.

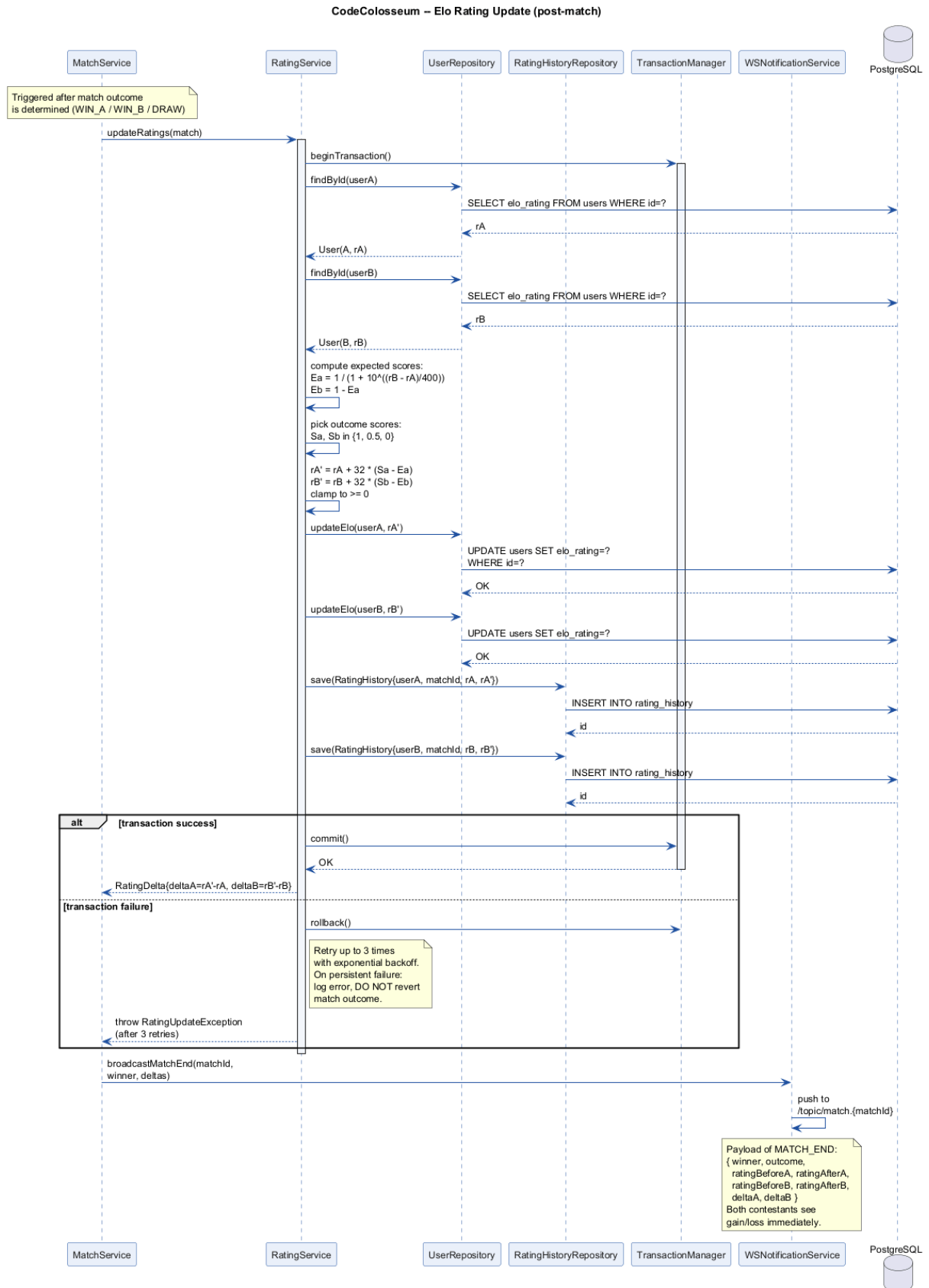


Figure 9: Sequence Diagram — Elo Rating Update

Diagram explanation. `RatingService.updateRatings()` is called by `MatchService` immediately after the match outcome is determined. The service opens a database transaction and retrieves the current Elo ratings of both contestants from `UserRepository`. It computes the expected scores using the standard Elo formula ($E_A = 1/(1+10^{(R_B-R_A)/400})$), applies the K=32 update ($R'_A = R_A + 32(S_A - E_A)$), and clamps results to 0. Both updated ratings are written back to the `User` rows, and two `RatingHistory` records (pre-rating, post-rating, matchId, timestamp) are inserted, all within the same atomic transaction. If the transaction fails, it is retried up to three times; on persistent failure the error is logged without reverting the match outcome. The rating deltas are included in the `MATCH_END` WebSocket payload so both contestants see their gain or loss immediately on the post-match screen.

5 Data Model

This section describes the logical and physical data model. The relational schema is PostgreSQL as the backend. Foreign key constraints and unique indices are defined at the database level.

5.1 Entity Descriptions

users Stores all registered accounts. Primary key: `id` (BIGINT, auto-increment). Unique constraints on `username` and `email`. The `password_hash` column stores the BCrypt output; plaintext is never persisted. `role` is an enum: `CONTESTANT` or `ADMIN`. `elo_rating` defaults to 1000 and has a CHECK constraint ≥ 0 . `total_matches` is a denormalised counter maintained by `MatchService` to avoid a full COUNT query on the leaderboard.

problems Stores problem proposals and approved problems. `status` is an enum: `PENDING`, `APPROVED`, `REJECTED`. `proposer_id` is a FK to `users`. `reviewer_id` is a nullable FK to `users` (the approving/rejecting administrator). `time_limit_seconds` and `memory_limit_mb` carry the resource constraints passed to Judge0.

test_cases Child of `problems` via FK `problem_id` (CASCADE DELETE). Stores `stdin` and `expected_output` as TEXT. These rows are server-side only; no API endpoint returns them to contestants.

matches Records each match session. `contestant_a_id` and `contestant_b_id` are FKs to `users`. `problem_id` is a FK to `problems`. `status` is `ACTIVE`, `COMPLETED`, or `CANCELLED`. `outcome` is `WIN_A`, `WIN_B`, or `DRAW` (nullable until completed). `started_at` and `ended_at` are `TIMESTAMP WITH TIME ZONE`.

submissions Child of `matches`. Records `user_id`, `match_id`, `language` (enum), `source_code` (TEXT), `verdict` (enum: `AC`, `WA`, `TLE`, `MLE`, `RE`, `CE`, `JUDGE_ERROR`), `test_cases_passed` (INT), and `submitted_at`.

rating_history Append-only log. Each row stores `user_id`, `match_id`, `rating_before`, `rating_after`, and `recorded_at`. Two rows are inserted per completed match (one per contestant) within the same transaction.

5.2 ERD (Entity Relationship Diagram)

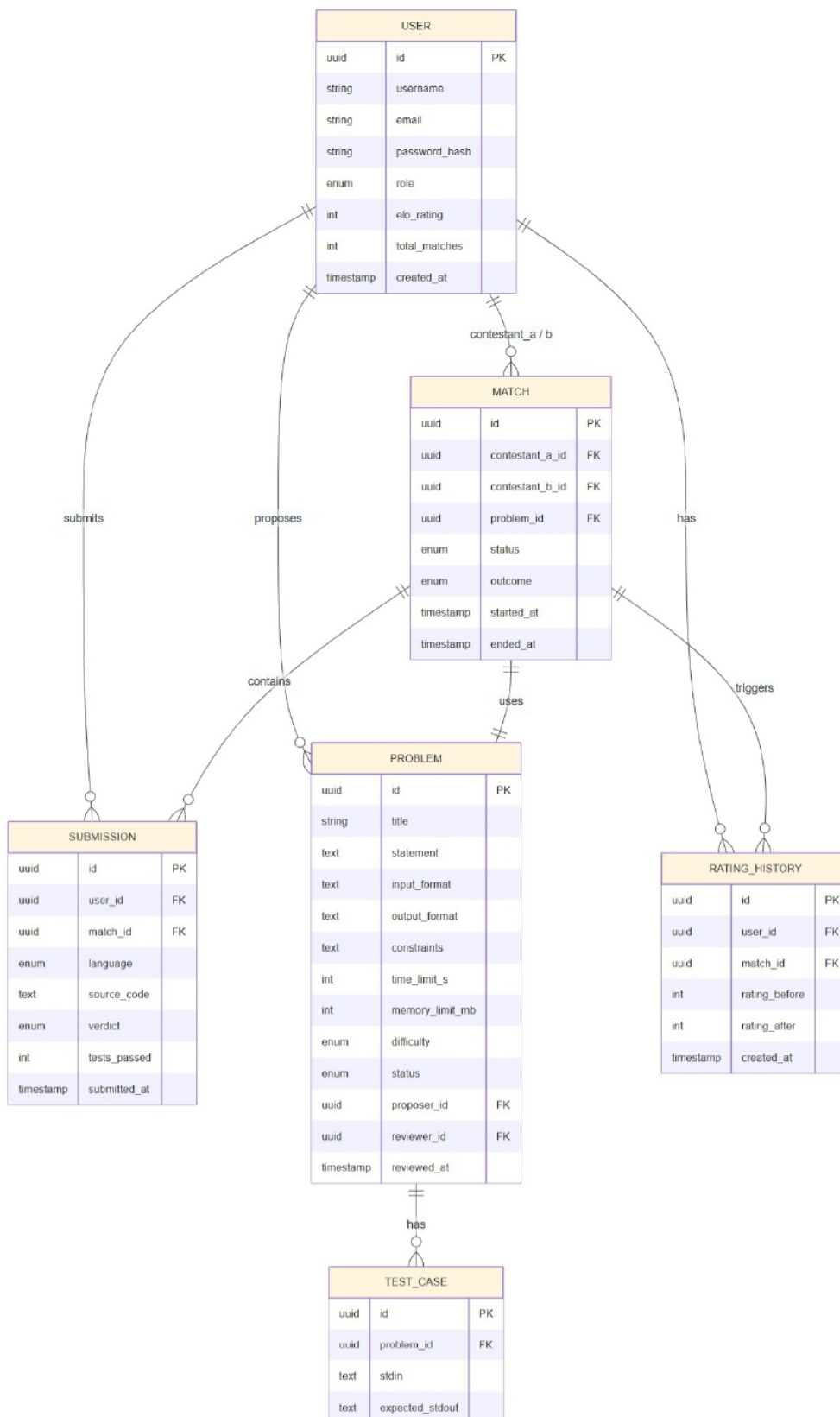


Figure 10: Entity Relationship Diagram

5.3 Entity-Relationship Overview

From	Card.	To	Notes
User	0..*	Problem	A user proposes many problems
Problem	3..*	TestCase	At least 3 test cases required for approval
Match	2	User	Exactly two contestants per match
Match	1	Problem	One problem per match
Match	0..*	Submission	Many submissions per match
Submission	1	User	Each submission belongs to one user
RatingHistory	1	User	One rating record per user per match
RatingHistory	1	Match	Links rating change to a specific match

6 Restrictions, Limitations, and Constraints

No server-side code execution. User-submitted code must never be run on the application server. This is enforced architecturally by routing all evaluation through `JudgeService` to the Judge0 sandbox. Violation would expose the server to arbitrary code execution.

Single-node deployment. The matchmaking queue is in-memory. The system does not support horizontal scaling in its initial form. A Redis-backed distributed queue would be required for multi-instance deployments.

Judge0 availability. Match functionality is contingent on Judge0 uptime. If Judge0 is unavailable, submissions will fail gracefully (`JUDGE_ERROR`) but the match will not be voided. Prolonged Judge0 downtime renders matches unresolvable by correct submission.

In-memory timer. The 30-minute match timer is maintained by the Spring backend. A server restart during an active match will cause timer state to be lost. No persistence of in-flight match timers is implemented.

Problem pool minimum. The matchmaking feature must not be made available to users until at least 10 approved problems exist in the pool (OR-4). This is an operational constraint, not a code-enforced one.

Browser compatibility. The frontend must function on the current stable release of Chrome, Firefox, and Edge. Internet Explorer and mobile browsers are explicitly out of scope.

No real-time in-match chat. WebSocket channels for the match carry only system-generated events (verdicts, timer, match outcome). No user-to-user messaging is implemented.

7 Conclusion

This document has defined the complete software design for CodeColosseum. The chosen architecture — a layered Spring Boot backend serving a React SPA over REST and WebSocket, with PostgreSQL for persistence and Judge0 for sandboxed code execution — directly satisfies all functional and non-functional requirements stated in the SRS while remaining implementable by a three-person team within one academic semester.

The three most consequential design decisions — delegating code execution to Judge0, using standard Elo with $K=32$ for rating, and applying partial test-case scoring for tie-breaking — are each motivated by a trade-off between implementation simplicity and competitive fairness, and are documented fully in the Decision Log (Appendix A). Each decision has been structured to minimise technical debt: execution is hidden behind an interface, the K-factor is configurable, and match resolution logic is isolated in a single service class.

The design provides a clear extension path. A custom Docker-based judge can replace Judge0 without changes outside `Judge0ServiceImpl`. A Redis queue can replace the in-memory queue to enable horizontal scaling. The Elo algorithm can be upgraded to Glicko-2 within `RatingService` alone. These paths are documented in the Decision Log and are deliberately preserved by the interface-driven architecture.

A Decision Log

The following decision log entries are reproduced from the SRS Appendix and supplemented with design-phase detail where relevant.

A.1 Decision Log 1: Code Execution Strategy

Decision	Delegate all code execution to the Judge0 REST API in the initial implementation.
Alternatives	(1) Build a custom Docker-based sandbox from scratch. (2) Use Judge0 API (selected).
Rationale	Building a custom sandbox with adequate security, multi-language support, and resource limiting would consume a disproportionate share of the available development time. Judge0 provides a production-grade, well-tested sandbox with a straightforward REST interface and broad language support including Java, Python 3, and C++17.
Trade-offs	Introduces an external runtime dependency. System availability for match functionality is partially contingent on Judge0 uptime and rate limits.
Risk mitigation	Execution logic is encapsulated behind the <code>JudgeService</code> interface (NFR-11). Replacing Judge0 requires only a new implementation class.

Technical debt	The custom Docker-based runner is documented as a future option for a subsequent iteration if the platform grows beyond the academic context.
-----------------------	---

A.2 Decision Log 2: Rating Algorithm

Decision	Use the standard Elo algorithm with a fixed K-factor of 32 for all rating updates.
Alternatives	(1) Simple win/loss points system (+10 / -10). (2) Glicko-2 rating system. (3) Standard Elo with K=32 (selected).
Rationale	A simple points system does not account for opponent strength and is therefore unfair. Glicko-2 is significantly more complex to implement correctly. Standard Elo is well-understood, widely used in competitive contexts, and produces meaningful and interpretable ratings at low implementation cost.
Trade-offs	K=32 can produce volatile rating swings for users with few matches. Elo does not model rating reliability (unlike Glicko-2).
Risk mitigation	The K-factor is a configurable parameter, not a hard-coded constant. Rating logic is isolated in <code>RatingService</code> and can be replaced without modifying any other component.
Technical debt	Migrating to Glicko-2 is documented as a future improvement.

A.3 Decision Log 3: Match Winner Determination

Decision	When no contestant achieves AC before time expires, the winner is the contestant who passed the greatest number of test cases across all submissions. In case of a tie, the match is recorded as a draw.
Alternatives	(1) Always draw if no AC. (2) Partial scoring (selected). (3) Penalty system.
Rationale	A pure “first AC or draw” rule penalises contestants who make substantial partial progress. Partial test-case scoring rewards productive effort, reduces uninformative draws, and better reflects relative skill.
Trade-offs	Requires tracking and comparing per-submission test case pass counts, adding modest complexity to match resolution logic. Wrong answers are not penalised, which slightly reduces accuracy.
Risk mitigation	Resolution logic is isolated entirely within <code>MatchService</code> and does not affect any other component.

Technical debt	None identified.
-----------------------	------------------

A.4 Decision Log 4: Persistence Strategy

Decision	Avoid the use of ORM frameworks (JPA/Hibernate); utilize Spring JDBC Template for direct SQL execution.
Alternatives	(1) Spring Data JPA/Hibernate. (2) Raw JDBC / JDBC Template (selected).
Rationale	To ensure maximum transparency and performance, the team opted against ORM abstractions. Direct SQL allows for precise optimization of complex joins (e.g., matching users with problems they haven't seen) and avoids the "hidden" query overhead and lazy-loading issues common in JPA. This approach also aligns with the pedagogical goal of maintaining a clear understanding of the relational schema and transaction boundaries.
Trade-offs	Increases the volume of "boilerplate" code. Changes to the schema require manual updates to all corresponding SQL strings in the Repository layer.
Risk mitigation	The Repository layer is strictly isolated. All SQL is centralized within these classes to ensure that schema changes do not leak into the Service or Controller layers. Extensive use of Spring's <code>JdbcTemplate</code> handles resource management (connection closing) and exception translation.
Technical debt	Higher maintenance effort for the Repository layer if the domain model grows.

A.5 Decision Log 5: Real-time Communication Protocol

Decision	Use STOMP over WebSockets for real-time match events.
Alternatives	(1) HTTP Long Polling. (2) Server-Sent Events (SSE). (3) WebSockets (STOMP) (selected).
Rationale	Competitive programming requires bidirectional, low-latency communication (e.g., immediate notification when an opponent submits). Long polling is too resource-heavy and slow. While SSE is lighter, WebSockets with STOMP provides a robust sub-protocol for "topic" subscriptions, making it much easier to isolate match data so User A doesn't see User B's private verdict events.

Trade-offs	WebSockets maintain an open TCP connection, increasing server memory usage. Requires specialized handling for connection drops/re-connections during a match.
Risk mitigation	Implementation of a heartbeat mechanism to detect stale connections and a 5-minute "re-entry" window for contestants who disconnect (FR-4.4).
Technical debt	None; standard practice for real-time competitive platforms.

A.6 Decision Log 6: Authentication Mechanism

Decision	Use stateless JWTs (HMAC-SHA256) carried in the <code>Authorization: Bearer</code> header.
Alternatives	(1) Server-side HTTP sessions. (2) OAuth 2.0 via external provider. (3) JWT (selected).
Rationale	Sessions require sticky load balancing or a shared session store, both unjustified overhead at our scale. OAuth introduces an external dependency chain (provider availability, redirect handling) disproportionate to our user-management needs. JWTs are self-contained, validate without a DB round-trip, attach cleanly to both REST and WebSocket upgrades, and are natively supported by Spring Security.
Trade-offs	Tokens cannot be revoked before expiry without a blocklist. Client-side storage has a known XSS/C-SRF trade-off regardless of location.
Risk mitigation	24-hour token lifetime; signing secret externalised to configuration; the same validator is reused for WebSocket upgrades.
Technical debt	A Redis-backed blocklist and refresh-token rotation are documented as future improvements.

A.7 Decision Log 7: Matchmaking Queue Implementation

Decision	Implement the queue as an in-memory <code>ConcurrentLinkedQueue</code> scanned every 1000 ms by a <code>@Scheduled</code> task.
Alternatives	(1) PostgreSQL queue with <code>SELECT ...FOR UPDATE SKIP LOCKED</code> . (2) Redis sorted set. (3) In-memory queue (selected).

Rationale	A DB-backed queue couples an ephemeral concern to a durable store and generates write load on every scan. Redis would scale horizontally but adds a second infrastructure component unjustified at tens of concurrent users. The in-memory queue meets the functional requirement with zero extra dependencies and scans in under 1 ms per cycle at the target load.
Trade-offs	Not durable (server restart empties it) and not replicable (blocks horizontal scaling; see section 6).
Risk mitigation	All queue access goes through <code>MatchmakingService</code> ; a future switch to Redis is a localised change. The frontend treats queue state as transient, so restart loss is visible but not destructive.
Technical debt	Migration to Redis is the first prerequisite for multi-instance deployment.

A.8 Decision Log 8: Matchmaking Proximity Threshold and Scan Interval

Decision	Pair contestants within ± 200 Elo, scanned every 1000 ms.
Alternatives	(1) Stricter ± 100 threshold. (2) Dynamic threshold widening with wait time. (3) Event-driven matching on enqueue. (4) Fixed $\pm 200 + 1000$ ms scan (selected).
Rationale	A tighter threshold risks indefinite waits at the low population expected during evaluation. Dynamic widening is fairer under arbitrary load but requires per-entry timer state. Event-driven matching creates races between concurrent enqueues; a periodic scan linearises decisions trivially. Fixed ± 200 produces fair pairs at the starting Elo distribution (all users begin at 1000) while keeping the implementation compact.
Trade-offs	At low population some contestants may wait indefinitely with no escalation. The 1000 ms scheduler imposes a floor on pairing latency even when both contestants are already queued.
Risk mitigation	Both values are externalised to configuration (<code>matchmaking.elo-threshold</code> , <code>matchmaking.scan-interval-ms</code>) and can be tuned without a redeploy. The pairing predicate is isolated in a single method.
Technical debt	A dynamic-widening threshold is documented as the preferred upgrade at steady-state volume.

A.9 Decision Log 9: Frontend Framework

Decision	Build the frontend as a React 18 SPA bundled with Vite.
Alternatives	(1) Vue 3. (2) Angular 17. (3) Svelte. (4) Server-rendered Thymeleaf. (5) React 18 (selected).
Rationale	Server-rendered templates were rejected first: the UI is intrinsically interactive (live timer, WebSocket-driven state). Among SPAs, Angular's module/DI system is disproportionate for a three-person team, and Svelte has a thinner ecosystem for our actual dependencies. React was chosen because (a) mature libraries exist for every feature we need (<code>@stomp/stompjs</code> , <code>@monaco-editor/react</code> , React Router); (b) all three team members had prior React exposure; and (c) the React + Vite pipeline is simple to demonstrate to evaluators.
Trade-offs	React is unopinionated about state management and folder structure, forcing additional team decisions that Angular or SvelteKit would prescribe.
Risk mitigation	State kept simple (<code>useState</code> + <code>useContext</code> , no external store). WebSocket logic centralised in a reusable <code>useMatchSocket</code> hook. Naming and structural conventions documented in the repository README.
Technical debt	Migration to TypeScript documented as a recommended next step.